

ECOM EXPERTS

Sonda PrestaSentinel

Manual de usuario e instalacion

sentinel_agent.php 1.1 · 03/09/2026

Contenido

1. Qué hace y para quién	
2. Compatibilidad	
3. Instalación	
3.1 Descargar la sonda ya configurada (recomendado)	
3.2 Instalación manual	
3.3 Qué crea y qué pasa al desinstalarla	
4. Configuración	
Cómo elegir un buen token	
5. Cómo se usa, día a día	
5.1 Comprobar que está viva (ping)	
5.2 Saber la versión exacta de PrestaShop (ps_version)	
5.3 Saber la versión de un módulo (module_version)	
5.4 Mirar el crontab del servidor (crontab)	
5.5 Ver qué hay dentro de la carpeta de un módulo (scan_module)	
5.6 Buscar puertas traseras (core_integrity)	
6. Qué ve el cliente en la tienda	
7. Preguntas frecuentes y problemas	
8. Seguridad	
8.1 Qué se ha revisado	
8.2 Lo que ya estaba bien	
8.3 Qué se ha arreglado en esta revisión	
8.4 Qué queda pendiente	
8.5 Recomendaciones de operación	
9. Ficha técnica	
Protocolo	
Acciones	
Límites incorporados	
Patrones que busca core_integrity	
Lado del panel	

Lo que no tiene

1. Qué hace y para quién

La sonda es un único fichero PHP que se sube a la raíz de una tienda PrestaShop y que permite al panel **PrestaSentinel** preguntarle a esa tienda cosas que desde fuera no se pueden saber. PrestaSentinel es el sistema de vigilancia de seguridad con el que se controlan varias tiendas de clientes a la vez: comprueba versiones, busca vulnerabilidades conocidas y avisa cuando algo huele mal. El problema es que desde Internet, a una tienda bien configurada, casi no se le puede sacar información: la versión exacta de PrestaShop está oculta, las versiones de los módulos no se publican y del servidor no se ve nada.

La sonda resuelve eso. Instalada dentro de la tienda, contesta a cinco preguntas concretas: qué versión de PrestaShop hay realmente, qué versión tiene un módulo determinado, qué hay en el crontab del servidor, qué ficheros hay dentro de la carpeta de un módulo, y qué ficheros PHP del sitio contienen patrones típicos de puerta trasera. Con eso, el panel deja de adivinar y pasa a saber: detecta que una tienda está en 8.1.5 y le faltan parches, o que alguien ha dejado un `eval(base64_decode(...))` colgado en una carpeta de módulos, o que hay una tarea programada que descarga un fichero de un dominio extraño.

La sonda solo lee. No escribe, no borra, no modifica nada y no toca la base de datos de la tienda. No tiene pantalla ni interfaz: no es una página que se visite, es un punto final que solo responde a peticiones firmadas criptográficamente por el panel. Un visitante que la encuentre por casualidad recibe un `403 forbidden` y nada más.

Está pensada para el técnico que administra las tiendas, no para el comerciante. Se instala una vez por tienda, se olvida, y se retira cuando el servicio de vigilancia termina.

2. Compatibilidad

- **No es un módulo de PrestaShop.** Es un fichero PHP suelto e independiente: no se instala desde el back-office, no aparece en la lista de módulos, no registra hooks y no crea tablas. Por eso todo lo relativo a instalación en tienda, pantalla de configuración y capturas del back-office no aplica.
- **PrestaShop:** cualquier versión, porque no usa ni una sola clase de PrestaShop. La detección de versión reconoce los formatos de 1.6 (`define('_PS_VERSION_', ...)` en `config/autoload.php`), 1.7 y 8.x (`const VERSION` en `app/AppKernel.php`) y 9.x (`src/Kernel.php`).
- **PHP:** comprobado sin errores de sintaxis en **7.4, 8.1, 8.2, 8.3 y 8.4**. El mínimo real es **PHP 5.6**, por el uso de `hash_equals()`. Probado en ejecución sobre PHP 8.2.
- **Dependencias:** ninguna. Ni Composer, ni librerías, ni extensiones fuera del núcleo de PHP (`hash`, `json`, `SPL` y `fnmatch`, todas presentes de serie).
- **Sistema operativo:** funciona en Linux y en Windows. La acción `crontab` solo devuelve datos en Linux, porque es donde existe el mandato `crontab` y los ficheros `/etc/crontab` y `/var/spool/cron/crontabs/`. En Windows devuelve la lista vacía, sin error.
- **Multitienda:** indiferente. La sonda trabaja sobre el sistema de ficheros de la instalación, que es común a todas las tiendas de un mismo PrestaShop.
- **Idiomas:** no aplica. La sonda no muestra texto a ninguna persona; devuelve JSON con claves fijas en inglés (`ok`, `version`, `lines`, `files`, `modified`, `error`).

3. Instalación

La sonda se instala en dos sitios: el fichero en la tienda del cliente y los datos de conexión en la ficha de esa tienda dentro del panel.

3.1 Descargar la sonda ya configurada (recomendado)

En el panel de PrestaSentinel, en la ficha del cliente, existe la acción de descarga de la sonda. El panel hace tres cosas por ti:

1. Si el cliente todavía no tiene token, **genera uno aleatorio** de 24 bytes con `random_bytes()` (192 bits) y lo guarda en su ficha.
2. Propone un nombre de fichero difícil de adivinar, del tipo `sentinel_1a2b3c4d.php` (las ocho primeras letras del MD5 del token).
3. Entrega el fichero **ya con el token dentro**, sustituyendo la línea de la constante.

La descarga queda registrada en la auditoría del panel como `download_agent`.

3.2 Instalación manual

Si prefieres hacerlo a mano:

1. Copia `sentinel_agent.php` y **renómbralo** a algo que nadie adivine, por ejemplo `sentinel_9f83.php`. No lo dejes con su nombre original.
2. Abre el fichero y cambia la línea de configuración: `php define('SENTINEL_AGENT_TOKEN', 'CAMBIA-ESTE-TOKEN-SECRETO');` por un token largo y aleatorio. **Mientras esa constante siga con el valor de fábrica, la sonda rechaza absolutamente todas las peticiones**, incluidas las bien firmadas: es una protección deliberada para que un despiste no deje una sonda abierta.
3. Sube el fichero **a la raíz de la tienda**, junto a `index.php`. Tiene que estar ahí y no en una subcarpeta, porque toma la raíz de la tienda de su propia ubicación (`__DIR__`) y desde ahí busca `modules/`, `themes/`, `override/`, `app/` y demás.
4. En el panel, en la ficha de la tienda, rellena los dos campos: - **URL de la sonda**: la dirección completa, por ejemplo `https://latienda.com/sentinel_9f83.php`. - **Token de la sonda**: exactamente el mismo texto que pusiste en la constante.
5. Comprueba que responde. Si el panel puede hablar con ella, la ficha del cliente pasa a mostrar la sonda como configurada y la detección de versión empieza a usarla.

3.3 Qué crea y qué pasa al desinstalarla

No crea nada. Ni tablas, ni carpetas, ni ficheros de registro, ni entradas en la base de datos de la tienda, ni tareas programadas. Es un fichero y nada más.

Desinstalar es borrar el fichero. No deja rastro. Conviene además vaciar los campos «URL de la sonda» y «Token de la sonda» en la ficha del cliente del panel, para que deje de intentar contactar con ella. **Cuando termine el servicio de vigilancia de una tienda, hay que borrar la sonda**: un fichero que lee el crontab y recorre el disco no debe quedarse para siempre en un sitio que ya no vigila.

4. Configuración

La sonda tiene **una sola opción de configuración**, y está dentro del propio fichero:

Dónde	Qué es	Qué poner
<code>sentinel_agent.php</code> , constante <code>SENTINEL_AGENT_TOKEN</code>	La clave secreta compartida con el panel. Es a la vez la contraseña y la clave con la que se firman las peticiones (HMAC-SHA256).	Un texto largo y aleatorio, distinto para cada tienda. Nunca el valor de fábrica.

Y dos campos en el panel, en la ficha de cada cliente:

Campo	Qué hace de verdad
URL de la sonda	Adónde manda el panel las peticiones POST. Si está vacío, el panel ni lo intenta: la sonda queda inactiva. Debe apuntar al fichero real, con el nombre que le hayas puesto.
Token de la sonda	Con este valor firma el panel cada petición. Si no coincide carácter por carácter con la constante del fichero, la sonda contesta <code>403</code> a todo y el panel se queda sin sonda, sin más aviso que la ausencia de datos.

Los dos campos van juntos: el panel solo activa la sonda cuando **ambos** están rellenos.

No hay pantalla de configuración ni capturas. La sonda no tiene interfaz: no es una página que se pueda abrir en el navegador ni un módulo con formulario en el back-office. Toda su configuración son la constante del fichero y esos dos campos del panel, que ya quedan descritos arriba. Por eso este manual no lleva imágenes.

Cómo elegir un buen token

Un token flojo tira por tierra toda la seguridad de la sonda. Lo más cómodo es dejar que lo genere el panel al descargar el fichero. Si lo generas tú, que tenga **al menos 32 caracteres aleatorios**, y **uno distinto por cada tienda**: si compartes el mismo token entre clientes, quien consiga el de una tienda tiene el de todas.

5. Cómo se usa, día a día

En el uso normal **no se toca**. El panel la interroga solo, dentro de los escaneos, y el técnico ve el resultado en la ficha del cliente. Estas son las cinco cosas que hace, en el orden en que el panel las usa.

5.1 Comprobar que está viva (ping)

Antes de nada el panel manda un `ping`. Si contesta, la sonda está bien instalada y el token coincide. Es la prueba que hay que hacer nada más instalarla. Devuelve la versión de la sonda, hoy `1.1`.

5.2 Saber la versión exacta de PrestaShop (`ps_version`)

Esta es la razón principal de instalarla. El panel intenta averiguar la versión por tres vías y **la sonda tiene prioridad sobre todas**: primero la sonda, si no hay sonda el FTP, y si no hay FTP la deducción por HTTP desde fuera, que es la menos fiable. La sonda lee directamente `app/AppKernel.php` , `src/Kernel.php` o `config/autoload.php` y saca la versión literal. Con el número exacto, el panel puede decir sin margen de error si a la tienda le faltan parches de seguridad.

5.3 Saber la versión de un módulo (`module_version`)

El panel pregunta por un módulo concreto y la sonda lee su `modules/<módulo>/config.xml` . Sirve para los avisos de vulnerabilidades que afectan a módulos de terceros: saber si el módulo instalado está por debajo de la versión corregida.

5.4 Mirar el crontab del servidor (`crontab`)

Devuelve las líneas del `crontab -l` del usuario y, si son legibles, las de `/etc/crontab` y `/var/spool/cron/crontabs/` . El panel las cruza con su lista de indicadores de compromiso. Es de las comprobaciones que más valen: una de las formas habituales de mantener el acceso tras entrar en un servidor es dejarse una tarea programada que vuelva a descargar la puerta trasera cada pocos minutos. Desde fuera eso es invisible.

5.5 Ver qué hay dentro de la carpeta de un módulo (`scan_module`)

Lista los ficheros de `modules/<módulo>/` que encajen con unos patrones. Sirve para detectar ficheros que no deberían estar ahí. **Ojo: solo mira el primer nivel de la carpeta**, no entra en las subcarpetas.

5.6 Buscar puertas traseras (`core_integrity`)

Recorre `modules/` , `themes/` , `override/` , `upload/` e `img/` , abre cada fichero `.php` y busca dentro los patrones típicos de un *webshell*: `eval(` , `base64_decode(` , `gzinflate(` , `str_rot13(` , `assert(` , `shell_exec(` , `system(` , `passthru(` y la palabra «eval» escondida en hexadecimal. Devuelve la lista de ficheros sospechosos.

Es una heurística, no una firma: da falsos positivos. Hay módulos legítimos que usan `base64_decode()` para cosas perfectamente normales. La lista es un punto de partida para mirar a mano, no una sentencia. Lo que sí es muy indicativo es encontrar un `.php` con esos patrones dentro de `img/` o de `upload/` , que son carpetas donde no debería haber código.

Para que no se coma el servidor, tiene dos toques: **8.000 ficheros por carpeta** y **200 resultados en total**, y de cada fichero solo lee los primeros 60 KB.

6. Qué ve el cliente en la tienda

Nada. La sonda no pinta nada en la tienda, no añade cabeceras ni scripts a las páginas, no ralentiza la navegación y no aparece en el back-office. Un visitante que llegase a adivinar la dirección del fichero recibiría un `403` con `{"error":"forbidden"}` y ninguna otra información: ni el nombre del producto, ni la versión de PrestaShop, ni siquiera la confirmación de que ahí hay una sonda y no un fichero cualquiera.

7. Preguntas frecuentes y problemas

El panel dice que la sonda no responde. ¿Qué miro? Por orden: (1) que la URL apunte al fichero real y se abra en el navegador — si da 404, el fichero no está donde crees; (2) que el token del panel y el del fichero sean idénticos, sin espacios de más al copiar; (3) que la constante del fichero ya no sea `CAMBIA-ESTE-TOKEN-SECRETO`; (4) que el reloj del servidor de la tienda esté en hora.

¿Por qué importa el reloj? Porque cada petición lleva una marca de tiempo firmada y la sonda rechaza todo lo que se desvíe **más de 5 minutos** de su propia hora. Es lo que impide que alguien capture una petición válida y la repita más tarde. Si el servidor de la tienda va desfasado, todas las peticiones parecen caducadas y la sonda contesta `403` aunque el token sea correcto.

Abro la URL en el navegador y me sale `forbidden`. ¿Está rota? No, está funcionando exactamente como debe. Un `403 forbidden` al entrar a mano es la respuesta correcta: solo el panel, que conoce el token, sabe firmar la petición.

¿Puede alguien usar la sonda para atacar la tienda? Sin el token, no: las seis acciones están detrás de la comprobación de firma. Con el token, podría leer la versión de PrestaShop, la de los módulos, el crontab y la lista de ficheros sospechosos. **Todo eso es solo lectura:** la sonda no tiene ninguna acción que escriba, borre o ejecute nada que le manden. Aun así, el token es un secreto y se trata como tal.

¿Consume recursos? Solo cuando el panel la llama. `ping`, `ps_version`, `module_version` y `scan_module` son instantáneos. `crontab` lanza un mandato del sistema. La pesada es `core_integrity`, que puede recorrer y leer miles de ficheros; en una tienda muy grande puede tardar y agotar el límite de 20 segundos que el panel espera. No es un fallo: significa que esa comprobación no se puede completar por HTTP en esa tienda.

El antivirus del hosting me marca el fichero. Puede pasar: el fichero contiene la palabra `shell_exec` y busca cadenas propias de *webshells*, así que algunos escáneres genéricos lo señalan por parecido. Es un falso positivo del escáner. Si el hosting lo bloquea, la alternativa es prescindir de la sonda y usar el acceso por FTP que el panel también admite.

¿Hay que actualizarla cuando actualizo PrestaShop? No. No depende de ninguna versión de PrestaShop.

Se me ha olvidado el token de una tienda. Está guardado en la ficha del cliente en el panel. Si aun así hay que cambiarlo, se cambia en los dos sitios a la vez: la constante del fichero y el campo del panel.

8. Seguridad

8.1 Qué se ha revisado

Se ha leído el fichero completo y se ha probado en ejecución contra una instalación de PrestaShop simulada (servidor PHP 8.2 real, peticiones firmadas de verdad con cURL), verificando las siete respuestas y los intentos de abuso. Se buscó: inyección SQL (no hay base de datos, no aplica), XSS (no hay HTML, la respuesta es JSON, no aplica), path traversal, autorización y firma, ejecución de mandatos, secretos en el código, fugas de información, cabeceras anti-caché, robustez en PHP 8 y respeto de las reglas del proyecto.

8.2 Lo que ya estaba bien

- La comparación de la firma usa `hash_equals()`, que es de tiempo constante: no se puede adivinar el token midiendo lo que tarda en contestar.
- La ventana temporal de 5 minutos limita mucho la reutilización de una petición capturada.
- La sonda se **autobloquea** mientras el token siga siendo el de fábrica, de modo que un fichero subido y no configurado no queda abierto.
- El nombre de módulo ya venía saneado con una expresión regular que solo deja letras, dígitos y guion bajo, lo que impedía el path traversal por ese camino.
- Ninguna acción escribe, borra ni ejecuta nada que llegue en la petición. No hay `eval`, ni `unserialize` de datos externos, ni ejecución de mandatos contruidos con datos del usuario: el único mandato es la cadena fija `crontab -l 2>/dev/null`.
- El fichero plantilla, con su token de fábrica, vive **fuera de la raíz web del panel** (la raíz es `public/`), así que no es descargable por nadie que pase por allí.
- El token que genera el panel usa `random_bytes()`, que es un generador criptográfico, con 24 bytes (192 bits) por tienda.

8.3 Qué se ha arreglado en esta revisión

Todo en `agent/sentinel_agent.php`. Cada cambio se ha comprobado con `php -l` en 7.4, 8.1, 8.2, 8.3 y 8.4, y con las peticiones reales del apartado 8.1.

1. Fatal de PHP 8 provocable en `scan_module` (el más serio). El parámetro `patterns` llegaba de la petición y sus elementos se pasaban tal cual a `fnmatch()` sin comprobar que fueran cadenas. Mandando un array anidado — `patterns[0][]=x` — se producía un `TypeError: fnmatch(): Argument #1 ($pattern) must be of type string, array given` que mataba la petición con un error 500. Se ha reproducido y confirmado en PHP 8.2. Ahora solo se aceptan cadenas no vacías de hasta 128 caracteres y como mucho 50 patrones; si no queda ninguno válido se usa el `*.php` de siempre. De paso se añadió un `break` al encontrar coincidencia, para no añadir el mismo fichero varias veces cuando encaja con varios patrones.

2. `scandir()` y `file()` sobre `false`. `scandir()` y `file()` devuelven `false` si no pueden leer, y `foreach` sobre `false` en PHP 8 es un aviso que ensucia la respuesta JSON y puede romper el `json_decode` del panel. Se han envuelto en `is_array($x) ? $x : array()`, como manda la norma del proyecto.

3. Recorrido de directorios que abortaba con una excepción sin recoger. En `core_integrity`, el `RecursiveDirectoryIterator` lanzaba una `UnexpectedValueException` en cuanto se topaba con una carpeta sin permisos de lectura, y como no estaba recogida se llevaba por delante toda la comprobación: el panel recibía un 500 en lugar de la lista de ficheros sospechosos. Justo el escenario más probable en un servidor comprometido, donde el atacante deja carpetas con permisos raros. Ahora se construye con `RecursiveIteratorIterator::CATCH_GET_CHILD`, que se salta las carpetas ilegibles y sigue, y además va dentro de un `try/catch` que pasa a la siguiente zona si algo falla.

4. Respuesta vacía cuando había bytes que no eran UTF-8. `json_encode()` devuelve `false` ante cualquier byte que no sea UTF-8 válido, y el código imprimía ese `false`, es decir, nada. Con rutas de fichero o líneas de crontab en otra codificación —cosa nada rara en un servidor viejo—, la sonda contestaba `200` con el cuerpo vacío y el panel lo interpretaba como que la sonda no funcionaba. Se ha centralizado la salida en una función `agent_out()` que usa `JSON_INVALID_UTF8_SUBSTITUTE` cuando existe (PHP 7.2 o superior) y que, si

aun así falla, devuelve un `{"error":"encode_failed"}` honesto en lugar de un cuerpo vacío. La comprobación con `defined()` mantiene la compatibilidad con PHP anteriores.

5. Contención de rutas en `agent_read()` (defensa en profundidad). La función que lee ficheros componía la ruta y la abría sin comprobar que el resultado siguiera dentro de la tienda. No era explotable, porque las dos llamadas que existen usan rutas fijas o el nombre de módulo ya saneado, pero bastaba con que alguien añadiera mañana una acción nueva para convertirlo en un path traversal. Ahora resuelve la ruta con `realpath()` y **rechaza cualquier fichero que no cuelgue de la raíz de la tienda**. Comprobado: pedir `module=../../../../../etc/passwd` devuelve `null`.

6. Conversiones de tipo con parámetros que no son cadenas. `action`, `sig` y `module` llegaban de la petición y se usaban directamente en concatenaciones y en `preg_replace()`. Mandándolos como arrays se provocaban avisos de «Array to string conversion» que se colaban en el cuerpo de la respuesta JSON. Ahora los tres se validan con `is_string()` antes de usarse, y `ts` con `is_scalar()`.

7. Cabeceras anti-caché incompletas. Solo se enviaba `Cache-Control: no-store`. La norma del proyecto exige las tres cabeceras para cualquier punto final que devuelva estado cambiante, y aquí importa de verdad: si un proxy o una CDN delante de la tienda cachea la respuesta de `core_integrity`, el panel puede estar viendo una foto de hace horas y dar por limpia una tienda que ya no lo está. Se han añadido `Pragma: no-cache` y `Expires: 0`. Se ha mantenido el `Content-Type: application/json` porque aquí quien consume la respuesta es `json_decode()` en PHP, no jQuery, así que no se da el problema de la doble decodificación.

También se ha subido el número de versión que devuelve `ping` de `1.0` a `1.1`, para poder distinguir en el panel las sondas ya endurecidas de las antiguas. El panel no comprueba ese número, así que **las sondas antiguas siguen funcionando**: el cambio es compatible hacia atrás en los dos sentidos.

8.4 Qué queda pendiente

Cuatro puntos que **no se han tocado a propósito**, porque arreglarlos exige cambiar también el lado del panel y eso rompería las sondas ya instaladas en tiendas de clientes. Se dejan documentados con su ubicación exacta.

1. La firma no cubre todos los parámetros. La firma se calcula solo sobre `acción|marca de tiempo`; `module` y `patterns` viajan **sin firmar**. Quien intercepte una petición válida puede, durante los 5 minutos que dura, reutilizar esa firma cambiando el módulo consultado o los patrones. El impacto real es bajo —lo peor que consigue es listar la carpeta de otro módulo, y el nombre está saneado—, pero lo correcto es firmar el conjunto completo de parámetros. Requiere cambiar a la vez `AgentProbe::sign()` en `src/Scan/AgentProbe.php` y la verificación de la sonda, y desplegar las dos cosas a la vez.

2. El panel no verifica el certificado TLS al hablar con la sonda. Es lo más importante de esta lista y **el arreglo está fuera de este fichero**. En `src/Support/Http.php`, `CURLOPT_SSL_VERIFYPEER` vale `false` salvo que se pase la opción `verify`, y `AgentProbe::call()` (línea 59 de `src/Scan/AgentProbe.php`) no la pasa. Resultado: las peticiones a la sonda van por HTTPS pero **sin comprobar contra quién se habla**, así que un intermediario podría capturar el token en claro y a partir de ahí interrogar la sonda a placer. Recomendación: pasar `'verify' => true` en esa llamada, aprovechando el `cacert.pem` que ya está en `C:\dev-tools\php\`, y dejar el modo sin verificar solo como excepción explícita por cliente.

3. Repetición de peticiones dentro de la ventana de 5 minutos. No hay número de un solo uso ni registro de firmas ya vistas, así que una petición capturada se puede repetir mientras no caduque. Como todas las

acciones son de solo lectura y no cambian nada, el daño se limita a volver a obtener la misma información. Arreglarlo bien pide almacenamiento en la tienda, que es justo lo que se quiso evitar al diseñar la sonda como un fichero suelto.

4. `get_current_user()` puede apuntar al usuario equivocado. En la acción `crontab`, la ruta `/var/spool/cron/crontabs/` se construye con `get_current_user()`, que devuelve el propietario del fichero, no necesariamente el usuario con el que corre PHP. En hostings donde no coinciden, ese crontab no se lee y la comprobación queda incompleta **sin avisar de ello**. No se ha cambiado porque el `crontab -l` anterior cubre el caso habitual y tocarlo cambiaría el comportamiento.

8.5 Recomendaciones de operación

- **Un token distinto por tienda**, y generado por el panel.
- **Nombre de fichero impredecible.** Nunca `sentinel_agent.php`.
- **Borrar la sonda al terminar el servicio.** Es la regla más importante de todas: un fichero que lee el crontab y recorre el disco no puede quedarse en una tienda que ya no se vigila.
- **Servir la tienda por HTTPS.** Con HTTP plano el token viaja en claro en cada escaneo.
- Si el hosting lo permite, restringir por IP el acceso al fichero de la sonda desde el servidor web, dejando pasar solo la IP del panel. Es una capa más, gratis.

9. Ficha técnica

Fichero único: `sentinel_agent.php` — 193 líneas, 7.592 bytes. **Ruta en el proyecto:** `check`
`hacks/agent/sentinel_agent.php` (fuera de `public/`, la raíz web del panel). **Ruta en la tienda:** raíz de PrestaShop, junto a `index.php`, renombrado. **Versión que declara:** `1.1` (en la respuesta de `ping`).

Protocolo

- **Método:** `POST` con `application/x-www-form-urlencoded`.
- **Parámetros comunes:** `action`, `ts` (marca de tiempo Unix), `sig`.
- **Firma:** `sig = hash_hmac('sha256', action + '|' + ts, TOKEN)`, comparada con `hash_equals()`.
- **Ventana temporal:** ± 300 segundos.
- **Respuesta:** JSON. `403` con `{"error": "forbidden"}` si la firma falla, la marca de tiempo está fuera de ventana o el token sigue siendo el de fábrica.
- **Cabeceras de respuesta:** `Content-Type: application/json; charset=utf-8`, `Cache-Control: no-store, no-cache, must-revalidate, max-age=0`, `Pragma: no-cache`, `Expires: 0`.

Acciones

Acción	Parámetros propios	Devuelve	Lee de
<code>ping</code>	—	<code>{"ok":true,"agent":"1.1"}</code>	nada
<code>ps_version</code>	—	<code>{"version":"8.2.7"}</code> o <code>null</code>	<code>app/AppKernel.php</code> , <code>src/Kernel.php</code> , <code>config/autoload.php</code> (64 KB máx. por fichero)
<code>module_version</code>	<code>module</code>	<code>{"version":"1.4.2"}</code> o <code>null</code>	<code>modules/<módulo>/config.xml</code> , etiqueta <code><version></code> , con o sin CDATA
<code>crontab</code>	—	<code>{"lines":[...]}</code>	<code>crontab -l</code> , <code>/etc/crontab</code> , <code>/var/spool/cron/crontabs/<usuario></code>
<code>scan_module</code>	<code>module</code> , <code>patterns[]</code>	<code>{"files":[...]}</code>	primer nivel de <code>modules/<módulo>/</code> , no recursivo
<code>core_integrity</code>	—	<code>{"modified":[...]}</code>	<code>.php</code> de <code>modules/</code> , <code>themes/</code> , <code>override/</code> , <code>upload/</code> , <code>img/</code>
cualquier otra	—	<code>{"error":"unknown_action"}</code> , HTTP 200	nada

Límites incorporados

Qué	Límite
Lectura por fichero en <code>agent_read()</code>	200.000 bytes (65.536 en las llamadas actuales)
Lectura por fichero en <code>core_integrity</code>	60.000 bytes
Ficheros recorridos por carpeta objetivo	8.000
Ficheros sospechosos devueltos	200
Patrones aceptados en <code>scan_module</code>	50, de 128 caracteres como mucho
Tiempo de espera del panel	20 s
Tamaño máximo de respuesta que acepta el panel	2 MB

Patrones que busca `core_integrity`

`eval(` , `base64_decode(` , `gzinflate(` , `str_rot13(` , `assert(` , `shell_exec(` , `system(` , `passthru(` y la secuencia hexadecimal de la palabra «eval».

Lado del panel

Pieza	Fichero
Cliente que llama a la sonda y firma las peticiones	<code>src/Scan/AgentProbe.php</code>
Activación de la sonda dentro del escaneo	<code>src/Scan/Scanner.php</code> (línea 116)
Generación del token y descarga del fichero configurado	<code>src/Web/Controller.php</code> , método <code>downloadAgent()</code>
Transporte HTTP	<code>src/Support/Http.php</code>
Campos del formulario	<code>views/clients.php</code> , <code>views/client_detail.php</code>
Columnas en base de datos	<code>agent_url</code> y <code>agent_token</code> en la tabla de clientes, <code>sql/schema.sql</code>

Lo que no tiene

Sin tablas propias, sin pestañas de administración, sin controladores, sin ficheros de configuración externos, sin tareas programadas propias, sin carpeta de registro y sin permisos especiales de fichero: le basta con permiso de lectura. Tampoco escribe ningún registro en la tienda; **la trazabilidad de quién descargó cada sonda está en la auditoría del panel**, con la acción `download_agent` .